ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

A PROJECT REPORT ON

SMART HOSPITAL MANAGEMENT SYSTEM (SmartCare)

Submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence & Data Science

Submitted to:

Rajasthan Technical University, Kota (Rajasthan)

Submitted By: Team Nobita

Student Name	RTU Roll No	Assigned Module
Aryan Daware	24EARAD020	Pharmacy, Laboratory and Nursing
Deepak Kumar	24EARAD035	Patient and Doctor Management
Devansh Mathur	24EARAD040	Authentication, Admin and Security
Devkinandan Kumawat	24EARAD042	Billing, Admission, Reports and Feedback
Hansraj Pandey	24EARAD055	Appointment and Consultation

Under the guidance of Er. Ram Babu Buri

Session: 2026-27

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO Industrial Area, Kukas, Jaipur, Rajasthan

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Certificate

This is to certify that the project report entitled Smart Hospital Management System (SmartCare), submitted by the following members of Team Nobita, is presented for evaluation under Project Based Learning in the Department of Artificial Intelligence & Data Science at Arya College of Engineering & I.T., Jaipur, for the academic session 2026–27.

Student Name	RTU Roll No
Aryan Daware	24EARAD020
Deepak Kumar	24EARAD035
Devansh Mathur	24EARAD040
Devkinandan Kumawat	24EARAD042
Hansraj Pandey	24EARAD055

The report describes the analysis, design and implementation of a role-based hospital management web application using Java, Jakarta Servlets, JSP, JDBC and MySQL. Its modules include patient and doctor management, appointments, pharmacy and laboratory workflows, nursing, admissions, billing, reports and feedback.

Project registration code: PBL2627-AI&DS-A-031

This certificate is to be completed and signed by the authorized academic representatives after evaluation.

Project Guide
Er. Ram Babu Buri

Head of Department
Dr. Ashok Kumar Kajla

Declaration

We, the members of Team Nobita, submit this report on Smart Hospital Management System (SmartCare) for academic evaluation under the Project Based Learning programme for the session 2026–27.

The report describes the project structure and functionality using the source code, database schema and work documentation supplied for this project. Project claims correspond to implemented or documented work. Where execution results have not been independently verified, they are described as design intent or testing recommendations rather than measured outcomes. Each team member confirms responsibility for the submitted academic work by signing below.

Student Name	RTU Roll No	Signature
Aryan Daware	24EARAD020	_____
Deepak Kumar	24EARAD035	_____
Devansh Mathur	24EARAD040	_____
Devkinandan Kumawat	24EARAD042	_____
Hansraj Pandey	24EARAD055	_____

Project registration code: PBL2627-AI&DS-A-031
Place: Jaipur
Date: _____

Acknowledgement

We express our sincere gratitude to our project guide, **Er. Ram Babu Buri** for his guidance and support throughout the development of Smart Hospital Management System. His help enabled us to organize the project into modules and connect software engineering concepts with a practical hospital workflow.

We thank the Department of Artificial Intelligence & Data Science and Arya College of Engineering & I.T., Jaipur, for providing the academic environment in which this Project Based Learning work was undertaken.

We also acknowledge the contributions of all Team Nobita members, whose module-level work covers authentication and administration, patient and doctor management, appointments and consultations, pharmacy and laboratory operations, nursing, admissions, billing, reporting and feedback.

We acknowledge the official documentation for Java, Jakarta Servlet, JSP, MySQL, Apache Tomcat, Maven, Docker and the related libraries used in the project. Reviewing the source code and preparing this report strengthened our understanding of layered web application architecture, database persistence, access control and deployment.

Team Nobita

Aryan Daware
Deepak Kumar
Devansh Mathur
Devkinandan Kumawat
Hansraj Pandey

Academic Year 2026-27

Abstract

Smart Hospital Management System (SmartCare) is a web-based application designed to organize common administrative and clinical workflows of a hospital in a single system. It provides role-specific interfaces for administrators, doctors, patients, nurses and receptionists. The project aims to reduce dependence on disconnected registers and repeated manual coordination by maintaining structured records for users, departments, patients, doctors, appointments, admissions, medical records, prescriptions, tests, medicine inventory, billing, payments, vital signs, feedback, notifications and audit activity.

The reviewed codebase is organized as a Maven web application. It uses Java 17, Jakarta Servlet and JSP APIs, JDBC, MySQL Connector/J, BCrypt password hashing and a MySQL relational database. Servlet controllers handle requests, filters participate in authentication and authorization, DAO classes encapsulate database operations, model classes represent entities, and JSP pages render the role-specific interfaces. The project includes a Dockerfile and is associated with a live Render URL supplied by the team.

The system modules are designed to support patient registration, doctor and department management, appointment booking, consultation-related records, prescriptions, laboratory workflows, nursing vital records, admissions and room allocation, billing and payments, feedback, reports and audit logs. The source includes input validation utilities and custom application/database exception classes. A database schema defines the principal entities and their relationships.

This report documents the problem context, objectives, requirements, architecture, modules, database design, algorithms, security considerations, testing approach, deployment and future improvements. The supplied source archive and work-log PDF are the primary evidence. The hosted URL returned a service-unavailable response during report preparation, so live acceptance of every workflow is not claimed; deployment and verification should be repeated when the service is available.

Keywords: Hospital Management System, SmartCare, Java, JSP, Servlets, JDBC, MySQL, Role-Based Access Control, Patient Records, Appointment Management, Billing.

Table of Contents

CERTIFICATE ii
DECLARATION iii
ACKNOWLEDGEMENT iv
ABSTRACT v
CHAPTER 1: INTRODUCTION 1
CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES 2
CHAPTER 3: EXISTING AND PROPOSED SYSTEMS 3
CHAPTER 4: FEASIBILITY STUDY 4
CHAPTER 5: REQUIREMENT ANALYSIS 5
REQUIREMENTS AND EXECUTION ENVIRONMENT 6
CHAPTER 6: SYSTEM ARCHITECTURE 7
CHAPTER 7: SYSTEM MODULES 9
CHAPTER 8: ROLE-BASED ACCESS CONTROL 10
CHAPTER 9: DATABASE DESIGN 11
CHAPTER 10: SOFTWARE DESIGN 13
CHAPTER 11: ALGORITHMS AND WORKFLOW 14
CHAPTER 12: SECURITY ANALYSIS 15
CHAPTER 13: TESTING STRATEGY AND TEST CASES 16
CHAPTER 14: RESULTS AND DISCUSSION 18
CHAPTER 15: SCOPE AND LIMITATIONS 19
CHAPTER 16: FUTURE SCOPE 20
CHAPTER 17: DEPLOYMENT AND SETUP 21
CHAPTER 18: PROJECT MANAGEMENT 22
REFERENCES 23
APPENDIX A: DATABASE TABLES 24
APPENDIX B: SOURCE STRUCTURE 25
APPENDIX C: USER GUIDE 26
APPENDIX D: SCREENS AND DEMONSTRATION CHECKLIST 30
APPENDIX E: PROJECT SUBMISSION CHECKLIST 31

Chapter 1: Introduction

Healthcare organizations coordinate several connected activities, from patient registration and appointment scheduling to clinical records, diagnostic tests, medicines, admission and payment. A digital information system can make these records easier to find and help staff coordinate tasks with fewer repeated entries.

1.1 Background

A hospital serves patients through multiple roles and departments. Reception staff register patients and arrange appointments; doctors review patient history and record consultations; nurses capture vital signs; laboratory staff process test workflows; pharmacy operations maintain medicine information; and administrators oversee users, departments and reports. If these activities are recorded in isolated files or registers, staff may need to repeat data entry and manually reconcile information.

1.2 Project Overview

SmartCare is a role-based Java web application intended to centralize these workflows. Its web layer consists of JSP pages and servlet controllers. DAO classes access MySQL using JDBC, while model classes represent domain objects such as Patient, Doctor, Appointment, Admission, Bill, Prescription and LabTest. Authentication and authorization filters are configured in the web application.

1.3 Aim

The main aim is to develop an academic prototype that organizes hospital records and common operations in a single browser-accessible application, with separate screens and permissions for different categories of users.

1.4 Report Organization

The following chapters explain the problem, objectives, feasibility, requirements, architecture, modules, access control, database schema, design, workflow, security, testing, results, limitations, future work, deployment and team project management.

Chapter 1 at a glance

SmartCare centralizes common hospital workflows in a browser-based Java application. JSP pages and servlet controllers handle the web layer, JDBC/DAO classes connect to MySQL, and role-based permissions separate staff functions.

Chapter 2: Problem Statement and Objectives

2.1 Problem Statement

Hospital workflows involve interconnected information. Patient and doctor availability, appointment details, medical history, prescriptions, laboratory tests, medicine stock, admissions and bills must remain associated with the correct patient and relevant staff. A fragmented process makes retrieval and coordination harder and increases the possibility of inconsistent records. The project addresses this organizational challenge through a centralized, role-based web application.

2.2 Proposed Solution

SmartCare provides role-specific dashboards and forms backed by a relational database. Application functions are divided among servlet controllers, service classes where present, DAO classes and JSP views. The schema includes related tables for users, clinical records, appointments, admissions, medicines, laboratory tests, billing and operational logs.

2.3 Objectives

- Maintain structured patient, doctor, nurse and receptionist records.
- Support user authentication and role-aware access to protected application routes.
- Manage appointments and related consultation information.
- Maintain medical records, prescriptions, laboratory test details and vital signs.
- Organize medicine catalogue and inventory information.
- Record admissions, room allocation, billing and payment information.
- Provide administrative summaries, reports, feedback and audit visibility.
- Use reusable Java classes and relational tables to improve maintainability.

2.4 Expected Outcomes

The expected outcome is a working educational prototype with a consistent application structure, persistent records and a demonstrable workflow across hospital roles. Quantified improvements in hospital efficiency, patient outcomes or cost savings have not been measured and are outside the evidence available for this report.

Objectives at a glance

The proposed system brings patient, doctor, appointment, clinical, laboratory, pharmacy, admission and billing records into one relational application. The report describes an academic prototype; measured efficiency or clinical outcomes are not claimed.

Chapter 3: Existing and Proposed Systems

3.1 Existing Approach

In a conventional or partially digitized workflow, records may be maintained in paper files, spreadsheets or separate departmental tools. Appointment details, diagnostic reports and billing entries can be stored in different places, requiring staff to reconcile information manually. This description is a general baseline for comparison, not a survey result about a particular hospital.

3.2 Proposed Approach

SmartCare consolidates the principal data entities in a MySQL schema and exposes workflows through a single Java web application. Users access functions according to their role, while the database links related records through identifiers and relational constraints.

Dimension	Conventional / fragmented approach	SmartCare approach
Patient information	May be spread across files and registers	Patient records represented in relational tables
Appointments	Separate schedule entries and manual coordination	Appointment records managed through application screens
Clinical records	May require searching separate folders	Medical records, prescriptions and lab information linked to patients
Medicine stock	Manual stock registers or separate lists	Medicine and medicine_inventory tables
Admission and rooms	Separate bed/room registers	Admission and room-related records
Billing	Manual calculations and separate payment records	Billing and payments represented in database tables
Access control	Depends on local process	Authentication and authorization filters plus role-oriented screens
Auditability	Depends on local logging	Audit log table and administration interface

3.3 Operational Significance

The principal benefit of centralization is traceability: records can be connected through database identifiers, and each role can focus on the information relevant to its work. The value of the system depends on accurate data entry, appropriate permissions, database backups and repeatable testing.

System comparison

The proposed workflow uses a centralized database and role-specific screens. This is intended to reduce duplicate entry and make linked records easier to retrieve while keeping access dependent on the user role.

Chapter 4: Feasibility Study

4.1 Technical Feasibility

The supplied project is a Maven WAR application built with Java 17, Jakarta Servlet 6.0, JSP 3.1 dependencies, MySQL Connector/J 8.3.0 and BCrypt 0.4. The project contains Java controllers, DAO classes, models, utilities, JSP views, CSS/JavaScript assets, a SQL schema and a Dockerfile. These components form a coherent basis for a Java web deployment.

4.2 Economic Feasibility

The stack uses widely available development tools and open-source libraries suitable for an academic prototype. Costs would mainly relate to developer time, hosting, database service, internet connectivity and maintenance. No verified budget or return-on-investment measurement was supplied.

4.3 Operational Feasibility

The system follows familiar hospital roles and workflows. A receptionist can register a patient or arrange an appointment, a doctor can review patient information and manage consultation-related records, a nurse can record vitals, and an administrator can oversee system data. Successful operation also requires staff training and clear responsibility for data quality.

4.4 Schedule Feasibility

The supporting work documentation records a 13-week project period from 6 July 2026 to 4 October 2026 and describes staged work from requirements and design through module coding, integration, testing, deployment and report preparation. The logged hours and weekly reports are included in the supplied PBL documentation.

4.5 Feasibility Summary

Area	Assessment basis	Status
Technical	Java/JSP/Servlet/JDBC/MySQL source and Maven build stack	Suitable for an academic prototype
Economic	Open-source stack; hosting and maintenance costs not quantified	Feasible subject to resource availability
Operational	Role-based workflows for hospital staff and patients	Requires training and accurate data
Schedule	13-week work log provided	Documented in supporting PBL log
Deployment	Dockerfile and supplied Render URL	Live service needs re-verification

Feasibility at a glance

The feasibility discussion considers technical implementation, operational use and project constraints. The system is designed as a student prototype using the Java web stack and a relational database.

Chapter 5: Requirement Analysis

5.1 Functional Requirements

ID	Requirement	Related project area
FR-01	Authenticate users and manage login/logout sessions	LoginServlet, LogoutServlet, AuthService
FR-02	Manage users, roles, departments and doctors	AdminServlet, UserDAO, DepartmentDAO, DoctorDAO
FR-03	Register and maintain patient information	PatientServlet, PatientDAO
FR-04	Create and manage appointments	AppointmentServlet, AppointmentDAO
FR-05	Maintain consultation-related records	DoctorServlet, MedicalRecordDAO
FR-06	Manage prescriptions and prescription items	PrescriptionServlet and related DAO/model classes
FR-07	Manage laboratory tests and their details	LabServlet, LabDAO
FR-08	Record patient vital signs	NurseServlet, VitalDAO
FR-09	Manage admissions and rooms	AdmissionServlet, AdmissionDAO, RoomDAO
FR-10	Maintain medicine and inventory information	MedicineServlet, MedicineDAO
FR-11	Create bills and record payments	BillingServlet, BillingDAO
FR-12	Collect feedback and maintain notifications	FeedbackDAO, Notification model
FR-13	Provide administrative reports and audit records	Admin screens, AuditDAO
FR-14	Validate user input and handle application/database errors	ValidationUtil and exception classes

5.2 Non-Functional Requirements

Attribute	Requirement interpretation
Usability	Consistent navigation, forms, tables and status information.
Maintainability	Separation of controllers, DAO classes, models, utilities and views.
Security	Authentication, role checks, password hashing and careful handling of sensitive records.
Integrity	Related records should reference valid users, patients, doctors and other entities.
Availability	The deployed service and database should be monitored and recoverable.
Performance	Common operations should respond within acceptable time under expected use; this must be measured.
Portability	A compatible JDK, servlet container and MySQL instance are required.
Privacy	Patient information should only be accessible to authorized users and used for legitimate purposes.

5.3 User Roles

The code and documentation identify the principal application roles as Admin, Doctor, Patient, Nurse and Receptionist. Permissions should be verified against the deployed configuration and servlet mappings before production use.

5.4 Execution Environment

Component	Project configuration
Language	Java 17

Chapter 5: Requirement Analysis (continued)

5.4 Execution Environment

Component	Project configuration
Language	Java 17
Web stack	Jakarta Servlet 6.0 and JSP 3.1
Database	MySQL; schema creates the SmartCare database
JDBC driver	MySQL Connector/J 8.3.0
Password hashing	BCrypt 0.4
Deployment artifacts	JSP, HTML, CSS and JavaScript; Dockerfile; web.xml
Build and packaging	Maven WAR packaging

Environment summary

The documented configuration uses Java 17, Jakarta Servlet 6.0, JSP 3.1, MySQL, Connector/J, BCrypt and Maven WAR packaging. Deployment files include web.xml and a Dockerfile.

Chapter 6: System Architecture

6.1 Architectural Overview

SmartCare follows a layered web architecture. A browser sends HTTP requests to the servlet container. JSP pages render the user interface, filters participate in authentication and authorization, servlet controllers coordinate actions, DAO classes perform JDBC operations, and MySQL stores the records. Model classes carry structured data between layers.

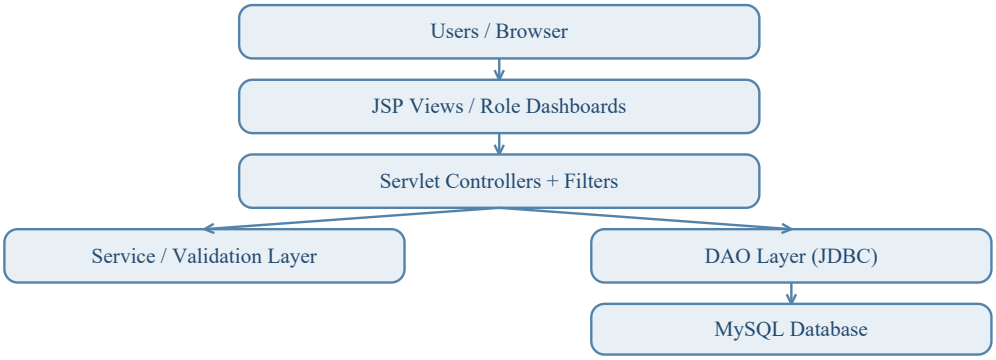


Figure 1. Conceptual layered architecture of SmartCare.

6.2 Presentation Layer

The presentation layer consists of JSP pages grouped by role and common tasks. Shared include files provide reusable navigation, headers, sidebars and footers. CSS and JavaScript support the layout and interactive elements.

6.3 Controller Layer

Servlet controllers receive requests, inspect parameters and sessions, invoke the relevant data-access or service component, and choose a response or view. The controllers include AdminServlet, PatientServlet, DoctorServlet, AppointmentServlet, NurseServlet, ReceptionistServlet, AdmissionServlet, LabServlet, MedicineServlet, BillingServlet, LoginServlet and LogoutServlet.

6.4 Data Access Layer

DAO classes isolate SQL operations from the JSP pages and most request-handling logic. Examples include PatientDAO, DoctorDAO, AppointmentDAO, AdmissionDAO, BillingDAO, LabDAO, MedicineDAO, PrescriptionDAO, RoomDAO, VitalDAO and AuditDAO.

6.5 Database Layer

The MySQL schema defines the main entities and their relationships. Foreign keys and unique constraints are used where specified in the schema. Database connection configuration should be supplied through environment-specific settings and must not expose credentials in source control.

6.6 Request Flow

- The user opens a page or submits a form in the browser.

Chapter 6: System Architecture (continued)

6.6 Request Flow

- The servlet container applies the configured filters to the request.
- A controller validates input and checks session/role context.
- The controller calls a service or DAO component to perform the operation.
- The DAO executes parameterized SQL and maps results to model objects.
- The controller forwards to a JSP or redirects to another route.
- The browser displays the resulting page or status message.

Request flow summary

A request passes through configured filters, controller validation and role/session checks, then the service or DAO layer. Parameterized SQL retrieves or updates records, and the controller returns a JSP view or redirects the browser.

Chapter 7: System Modules

7.1 Authentication, Administration and Security

This module supports login and logout, user and role administration, department configuration, administrative dashboards and audit visibility. PasswordUtil provides password hashing support through BCrypt, while AuthService coordinates credential validation. AuthenticationFilter and AuthorizationFilter are present to restrict protected areas.

7.2 Patient and Doctor Management

Patient and doctor modules maintain core records used by appointments and clinical workflows. Patient screens include profile, appointments, medical records, prescriptions, lab reports, bills/invoices and feedback. Doctor screens include a dashboard, appointments, consultation, patient profile, records, prescriptions and lab reports.

7.3 Appointment and Consultation

Appointment management coordinates patient requests and doctor schedules. The application includes appointment lists, appointment booking screens for patients and receptionists, and doctor-facing appointment and consultation pages. The database contains an appointments table to persist appointment information.

7.4 Pharmacy, Laboratory and Nursing

Medicine and inventory functions support medicine catalogue and stock records. Laboratory pages cover lab management, test details and reports. Nursing screens provide patient lists, vital recording and vital viewing. The schema contains medicines, medicine_inventory, lab_tests and patient_vitals tables.

7.5 Billing, Admission, Reports and Feedback

Admission functions organize inpatient records and room assignment. Billing pages support bill generation and review, while payment records are maintained separately. Administrative pages include billing, reports, rooms and audit views. Patient feedback is represented by a dedicated feedback DAO and page.

Architecture summary

The application separates browser views, servlet/controller logic, data access and domain models. This separation helps keep user interface code distinct from database operations and supports clearer module responsibilities.

Chapter 8: Role-Based Access Control

8.1 Purpose

Role-based access control helps ensure that each user can access only the operations needed for their responsibilities. In a hospital application, this is especially important because patient records and billing details may be sensitive.

8.2 Role Responsibilities

Role	Typical responsibilities represented by the project
Admin	Manage users, departments, doctors, rooms, reports, billing views and audit records.
Doctor	View assigned appointments and patient details; manage consultation-related records and prescriptions.
Patient	View profile and personal records, book appointments, review reports/bills and provide feedback.
Nurse	View patients and record or review vital signs.
Receptionist	Register patients, arrange appointments, manage patient-facing front-desk workflows and billing tasks.

8.3 Authentication and Authorization

Authentication confirms a user identity after credential checking. Authorization evaluates whether the authenticated user is allowed to access a route or operation. The project includes AuthenticationFilter and AuthorizationFilter classes, but each protected route and every data-changing operation should be verified through direct URL tests and negative-role tests.

8.4 Session Handling

A secure deployment should invalidate sessions on logout, set appropriate session-cookie flags, enforce inactivity timeouts and avoid exposing session identifiers. The current implementation should be reviewed against these controls before the application is used with real patient information.

8.5 Access-Control Verification

- A user without a session should not open protected pages directly.
- A patient should not be able to invoke administrative endpoints.
- A doctor should not be able to edit another role's administrative settings.
- Logout should invalidate the current session.
- Role restrictions should be enforced server-side, not only by hiding navigation links.

Chapter 9: Database Design

9.1 Database Overview

The supplied schema creates a MySQL database named smart_hospital. It contains tables for identity and access, hospital staff, patients, clinical workflows, inventory, finance and operational tracking. Relational tables allow a record to refer to another record through an identifier instead of duplicating all associated information.

9.2 Main Tables

Table	Purpose
roles	Defines application role information.
departments	Stores hospital departments.
users	Stores user account and authentication-related data.
doctors	Doctor-specific information associated with users and departments.
patients	Patient demographic and contact information.
nurses	Nurse-specific staff records.
receptionists	Receptionist-specific staff records.
rooms	Room or bed allocation details.
admissions	Inpatient admission details.
appointments	Appointment scheduling records.
medical_records	Clinical record information associated with patients.
medicines	Medicine catalogue.
medicine_inventory	Stock/inventory information.
prescriptions	Prescription header records.
prescription_items	Individual medicine items associated with a prescription.
lab_tests	Laboratory test details and status.
billing	Bills generated for patient services.
payments	Payment transactions associated with billing.
patient_vitals	Vital measurements recorded for patients.
feedback	Feedback supplied through the patient workflow.
audit_logs	Selected system activity and administrative audit information.
notifications	Application notification records.

9.3 Relationships

The schema is designed around common relationships: a patient can have multiple appointments, medical records, prescriptions, laboratory tests, bills and vital measurements; a doctor can be associated with appointments and clinical records; admissions can be associated with patients and rooms; a bill may have one or more payment records; and prescription headers can have multiple prescription items. The exact foreign-key definitions in schema.sql are authoritative for implementation details.

9.4 Data Integrity

Chapter 9: Database Design (continued)

9.4 Data Integrity

Primary keys provide stable record identifiers. Foreign keys, uniqueness constraints and not-null requirements can prevent invalid relationships and duplicate values where configured. Controllers should also validate user input before DAO operations, and transactions should be used when one business operation updates several related tables.

9.5 Privacy Considerations

The database may contain personal and health-related data. Production use requires restricted database accounts, encrypted connections where supported, backups, careful logging, retention rules and a clear policy for test data. Real patient information should not be used in demonstrations without proper authorization.

Database design notes

Primary keys identify records, while foreign keys and constraints help maintain valid relationships. Input validation and transactions are important when one operation changes multiple related records; privacy requires restricted access and careful handling of test data.

Chapter 10: Software Design

10.1 Object-Oriented Design

The project uses Java model classes to represent business entities such as Patient, Doctor, Appointment, Admission, Bill, Payment, Prescription, LabTest, Room and User. Encapsulating data in models makes it easier for controllers and DAO classes to exchange structured values.

10.2 DAO Pattern

The Data Access Object pattern separates database queries from request-handling code. A DAO method can retrieve or update records while the controller focuses on HTTP parameters, session context and response selection. This separation improves readability and makes database operations easier to review.

10.3 Service and Utility Classes

Service classes are present for authentication, appointments, admissions and billing. Utility classes include DBConnection, PasswordUtil and ValidationUtil. ApplicationException and DatabaseException provide named exception types for error handling.

10.4 Naming and Package Structure

Package / folder	Responsibility
controller	Servlet request handlers and application workflows.
dao	SQL access and persistence operations.
model	Java representations of domain entities.
service	Business-level coordination for selected workflows.
util	Database connection, password and validation utilities.
filter	Authentication and authorization checks.
exception	Application and database exception types.
webapp/*	JSP views, static resources, shared includes and error pages.
database/schema.sql	Database and table creation script.

10.5 Maintainability

The modular structure is a useful base for maintenance. Future refinements should standardize validation and error responses, avoid duplicated SQL, keep configuration outside source control, add automated tests and document each endpoint's role and expected parameters.

Design summary

The software design maps the documented requirements to application modules and data structures. Tables and interfaces should remain consistent with the model and with the role-based workflows described in the report.

Chapter 11: Algorithms and Workflow

11.1 Login Workflow

- Receive the login identifier and password.
- Validate that required fields are present.
- Retrieve the matching active account using a DAO query.
- Verify the submitted password against the stored password hash.
- On success, establish the authenticated session and route the user to the appropriate dashboard.
- On failure, return a clear message without revealing sensitive account details.

11.2 Appointment Workflow

- Patient or receptionist opens the appointment form.
- The form collects patient, doctor, date/time and relevant appointment details.
- The controller validates required values and applicable business rules.
- AppointmentDAO stores the appointment and returns the result.
- The dashboard/list displays the updated appointment status.

11.3 Patient Record Workflow

A patient record is created or selected, and the relevant clinical information is retrieved using DAO methods. The doctor or authorized staff member enters updates through a role-specific screen. The controller validates the request and persists the record. Subsequent screens retrieve the saved information by patient and record identifiers.

11.4 Admission and Billing Workflow

An admission workflow associates a patient with admission details and a room. Billing workflows assemble charge information and persist a bill. Payment records are maintained separately so that payment events can be associated with a bill. Exact billing rules, payment status transitions and concurrency behavior should be confirmed with dedicated test cases.

11.5 Pseudocode: Generic Create Operation

```
BEGIN
  receive request, session and required role
  validate required fields and allowed values
  if validation fails:
      return form with validation message
  open database operation
  execute parameterized insert/update operation
  if operation succeeds:
      commit and show success state
  else:
      rollback, log safely and show a generic error
END
```

11.6 Workflow Principles

All data-changing operations should validate the role, validate submitted values and use parameterized database access. Multi-table changes should use transactions and return a clear result to the user.

Chapter 12: Security Analysis

12.1 Password Protection

The project includes PasswordUtil and the jBCrypt library. BCrypt is designed for password hashing and includes a salt in the stored hash format. Passwords should never be stored as plaintext or included in application logs. Hashing parameters should be selected with current performance and security guidance in mind.

12.2 SQL Injection Prevention

DAO queries should use PreparedStatement parameters rather than concatenating user input into SQL strings. The code review should cover every query, including search, sort and pagination parameters, because dynamically constructed SQL can introduce risk if values are not allow-listed.

12.3 Authentication and Authorization

AuthenticationFilter and AuthorizationFilter provide a structure for controlling protected routes. Every servlet and action should be mapped to the correct roles, and authorization should be applied to the requested record as well as the page. Hiding a link in the interface is not a security boundary.

12.4 Input Validation

ValidationUtil is present for common checks. Validation should include required fields, length limits, formats, numeric ranges, date/time logic and allow-listed status values. Server-side validation errors should not disclose SQL statements, file paths or credentials.

12.5 Audit and Error Handling

AuditDAO and the audit_logs table support selected activity logging. Audit records should capture meaningful administrative actions while avoiding passwords, session identifiers and unnecessary medical details. Application and database exceptions should be logged with a correlation identifier and a safe user-facing message.

12.6 Deployment Security Checklist

- Keep database credentials in environment variables or protected configuration, not public source control.
- Use HTTPS for the hosted application and secure database connections where supported.
- Restrict database network access and use least-privilege credentials.
- Review session-cookie settings, timeouts, logout behavior and protected route mappings.
- Use synthetic records for demonstrations and avoid unnecessary personal health data in logs.
- Maintain backups and test restoration; review dependencies and deployment logs.

Chapter 13: Testing Strategy and Test Cases

13.1 Testing Approach

Testing should cover unit-level logic, servlet/controller behavior, DAO/database operations, role restrictions, user-interface behavior, integration between modules and the deployed environment. The supplied work documentation records testing and integration as project activities. This report does not invent pass/fail results for tests that were not independently executed during its preparation.

13.2 Functional Test Matrix

Test ID	Scenario	Expected result	Status in this report
TC-01	Login with valid active account	Session established and correct dashboard shown	To execute/confirm
TC-02	Login with incorrect password	Login rejected with safe message	To execute/confirm
TC-03	Open protected page without login	Request redirected or denied	To execute/confirm
TC-04	Patient opens admin route directly	Access denied	To execute/confirm
TC-05	Register patient with valid data	Patient record saved and visible	To execute/confirm
TC-06	Submit patient form with missing fields	Validation message; no invalid record saved	To execute/confirm
TC-07	Book appointment with valid details	Appointment stored and shown in list	To execute/confirm
TC-08	Record a medical entry as authorized doctor	Record saved and associated with patient	To execute/confirm
TC-09	Record patient vitals as nurse	Vital record saved and retrievable	To execute/confirm
TC-10	Create admission and assign room	Admission and room linkage remain consistent	To execute/confirm
TC-11	Generate bill and record payment	Billing/payment records remain associated	To execute/confirm
TC-12	Attempt SQL metacharacters in text input	Input handled safely; query structure preserved	To execute/confirm
TC-13	Logout then revisit protected page	Session no longer grants access	To execute/confirm
TC-14	Database unavailable during request	Safe error response and diagnostic logging	To execute/confirm
TC-15	Open hosted application and complete role workflow	Expected pages and persistence operate on deployment	Recheck; hosted URL unavailable during preparation

Testing focus

Test cases should cover valid and invalid inputs, linked records, authentication and role restrictions. Integration checks verify that related patient, appointment, prescription, admission and billing data remain consistent.

Chapter 13: Testing Strategy and Test Cases (continued)

13.3 Integration Testing

Integration tests should verify that patient records appear in appointment and clinical screens, that appointment records refer to valid patients and doctors, that prescriptions can be retrieved from the patient view, that admission and room records stay consistent, and that bills and payments remain correctly linked. Test data should be isolated from real operational records.

13.4 Regression Testing

After a fix, rerun the affected case and the key workflows that depend on the same tables or routes. Regression testing is particularly important when shared includes, filters, session handling or common DAO methods are changed.

13.5 Deployment Smoke Tests

After deployment, verify the home/login page, authentication, role-based routing, database connectivity, one representative read/write operation per major module, logout and error handling. The supplied live URL returned HTTP 503 during report preparation; therefore, live smoke-test completion is not asserted.

Regression and deployment checks

After a fix, rerun the affected case and dependent workflows. Deployment smoke tests should check login, role routing, database connectivity, representative read/write operations, logout and error handling.

Chapter 14: Results and Discussion

14.1 Implemented Project Structure

The supplied source archive contains Java controller, DAO, model, service, utility, filter and exception packages, along with JSP views for the major roles. It also contains the database schema, Maven configuration and Dockerfile. The module layout aligns with the five areas listed in the supporting team work log.

14.2 Functional Coverage Represented in Source

Area	Evidence in supplied project
Authentication	LoginServlet, LogoutServlet, AuthService and filters
Patients	PatientServlet, PatientDAO, doctor servlet, DAO/model classes
Appointments	AppointmentServlet and booking/list pages
Clinical records	MedicalRecordDAO, PrescriptionServlet, prescription and record pages
Nursing	NurseServlet, VitalDAO, vital-record pages
Laboratory / pharmacy	LabServlet, LabDAO and medicine/inventory pages
Admission and rooms	AdmissionServlet, AdmissionDAO and room/admission pages
Billing	BillingDAO, bill and invoice pages
Administration	AdminServlet and administrative dashboard, user, department, report and audit pages

14.3 Interpretation

The code organization demonstrates a broad educational implementation of hospital administration and clinical record workflows. However, the presence of a source file or screen does not alone prove that every use case is fully accepted in the deployed environment. A final demonstration should use the testing matrix in Chapter 13 and retain evidence for each confirmed result.

14.4 Evidence Boundary

The work-log PDF records a 13-week work period and team module assignments. The supplied archive was inspected for its source structure and schema. The deployment URL could not be fetched successfully at the time this report was prepared. Consequently, this report describes code-level structure and documented intent, not verified production readiness or measured hospital outcomes.

Chapter 15: Scope and Limitations

15.1 Scope

- User authentication and role-based navigation.
- Patient, doctor, nurse and receptionist records.
- Appointment booking and appointment lists.
- Medical records, prescriptions, laboratory information and vital signs.
- Medicine catalogue and inventory records.
- Admissions, room allocation, billing and payment records.
- Administrative reporting and audit-related screens.

15.2 Limitations

- Live deployment behavior could not be confirmed because the supplied URL returned a service-unavailable response during preparation.
- No independent load, penetration or formal acceptance test results were supplied.
- The project should be treated as an academic prototype until security, privacy, backup and operational requirements are reviewed.
- External integrations such as insurance providers, payment gateways, hospital information exchanges or diagnostic devices are not claimed unless separately implemented and verified.
- Real clinical deployment would require governance, data-protection review, user training, support processes and ongoing maintenance.

15.3 Intended Users

The system is intended as an academic demonstration of hospital information management workflows for administrators, doctors, patients, nurses and receptionists. Its use with real health information requires additional review and authorization.

Scope summary

The system scope is limited to the functions and configuration documented in this report. Operational rollout, external integrations and production-grade governance may require further validation beyond the academic prototype.

Chapter 16: Future Scope

16.1 Security Enhancements

Future versions can introduce multi-factor authentication for privileged roles, stronger password policy controls, explicit CSRF protection, centralized authorization rules, rate limiting and periodic dependency scanning. Security logging should be reviewed to ensure it records necessary events without leaking sensitive information.

16.2 Usability and Accessibility

The interface can be improved with responsive layouts, accessibility checks, clearer form errors, keyboard navigation, consistent status indicators and better support for small-screen devices. User testing with representative workflows can help prioritize interface changes.

16.3 Reporting and Analytics

Administrative dashboards could provide carefully defined counts and trends for appointments, admissions, laboratory workflow, medicine stock and billing. Any analytics should use accurate definitions and should not imply clinical decision support without suitable validation.

16.4 Reliability and Integration

Potential extensions include automated backups with restore drills, structured monitoring, deployment health checks, notification delivery, export tools, integration APIs and a formal migration process. External services should be added only with authentication, privacy controls and clear failure handling.

16.5 Quality Engineering

A future iteration should add automated unit and integration tests, seed data for reproducible demonstrations, CI build checks, a documented release process and acceptance evidence for every major user role.

Future work summary

Future improvements should be prioritized against demonstrated needs and verified with tests. Reporting, usability, security review and deployment validation can be refined without claiming features that have not been tested.

Chapter 17: Deployment and Setup

17.1 Deployment Overview

The project includes a Dockerfile and Maven WAR packaging. The supporting documentation associates the application with a Render-hosted URL and cloud MySQL. The exact environment variables, secrets and service configuration should be maintained in the hosting provider's protected settings rather than embedded in the report.

17.2 Local Setup Outline

- Install a compatible JDK (Java 17) and Maven.
- Install or provision a compatible MySQL server.
- Create the database by executing database/schema.sql.
- Configure database connection properties securely for the local environment.
- Build the WAR using Maven and deploy it to a compatible Jakarta Servlet container.
- Verify login, role-specific routes, connectivity and representative module workflows.
- Use synthetic data and avoid committing credentials or real patient records.

17.3 Build Command

```
mvn clean package
```

17.4 Containerization

The Dockerfile is included in the supplied archive. A container deployment should ensure the runtime servlet container is compatible with the Jakarta APIs used by the application, expose the configured application port, pass database configuration through environment variables and emit useful logs to the hosting platform.

17.5 Deployment Verification

The provided hosted URL is https://smart-hospital-management-system-fnb5.onrender.com/. It returned a 503 Service Unavailable response when checked for this report, so the deployment should be restarted or inspected and then smoke-tested before final academic submission. This observation is time-specific and does not prove the application is permanently unavailable.

17.6 Operational Checklist

Check	Purpose
Build	Confirm successful initialization and detect dependency issues.
Database	Confirm required tables and constraints exist.
Health/home page	Confirm connection settings and permissions.
Role workflow	Confirm web container serves the application and each role can perform its intended work.
Read/write	Confirm persistence and linked records.
Logs and error pages	Keep failures diagnosable without exposing secrets.
Backup/restore	Confirm recovery is practical.

Chapter 18: Project Management

18.1 Team Structure

Member	Assigned module	Logged hours in work document
Devansh Mathur	Authentication, Admin and Security	115.75
Deepak Kumar	Patient and Doctor Management	113.25
Hansraj Pandey	Appointment and Consultation	116.25
Aryan Daware	Pharmacy, Laboratory and Nursing	112.00
Devkinandan Kumawat	Billing, Admission, Reports and Feedback	116.75
Team Total	Five integrated modules	574.00

18.2 Timeline

Phase	Period described in work log	Main activity
1	6-12 July 2026	Team formation, guide selection and abstract
2	13-19 July 2026	Software Requirement Specification
3	20-26 July 2026	UML and system design
4	27 July-2 August 2026	Database design and project setup
5-8	3-30 August 2026	Models/DAO, services/servlets, JSP views, validation and security
9-10	31 August-13 September 2026	Module completion and integration
11	14-20 September 2026	Testing and role-based verification
12	21-27 September 2026	Deployment and bug fixing
13	28 September-4 October 2026	Report, presentation, demo and viva preparation

18.3 Collaboration

The team split the application into five major modules and integrated them into a unified project. The supplied work documentation records weekly reports and day-wise entries for the stated 13-week period. Module boundaries provide a clear basis for assigning responsibility while integration tests check that records and navigation work across modules.

18.4 Deliverables

- Project source code and Maven configuration.
- Database schema and application pages.
- Container/deployment artifacts.
- Project report, presentation and demonstration materials.
- Testing evidence and final academic submission materials.

References

(1) Oracle, Java Documentation. https://docs.oracle.com/en/java/

(2) Jakarta EE, Jakarta Servlet Specification and API Documentation. https://jakarta.ee/specifications/servlet/

(3) Apache Maven Project, Maven Guides. https://maven.apache.org/guides/

(4) MySQL, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

(5) Apache Tomcat, Tomcat Documentation. https://tomcat.apache.org/

(6) Mindrot, jBCrypt library. https://www.mindrot.org/projects/jBCrypt/

(7) Docker, Docker Documentation. https://docs.docker.com/

(8) Render, Web Services Documentation. https://render.com/docs/web-services

(9) SmartCare project source archive supplied for this report, including pom.xml, database/schema.sql, Java source files and JSP views.

(10) Smart Hospital Management System (SmartCare), Consolidated PBL Reports - Daily & Weekly Team Nobita, academic year 2026-27.

(11) Smart Hospital Management System hosted demonstration URL supplied by the project team: https://smart-hospital-management-system-fnb5.onrender.com/.

The project source archive and team work-log PDF are the primary project-specific references. External documentation is listed for the technologies used; references do not imply that every optional feature of those platforms is implemented in SmartCare.

References note

The reference list supports the technologies and standards discussed in the report. Before submission, verify that each citation is complete, relevant to the implementation and formatted consistently.

Appendix A: Database Tables

The supplied database/schema.sql file is the authoritative definition for the database. The table list below is a high-level index intended to help readers navigate the schema; exact column types, keys, indexes and constraints should be read from the SQL file.

Group	Tables
Identity and access	roles, users, departments
Staff	doctors, nurses, receptionists
Patient operations	patients, appointments, medical_records, prescriptions, prescription_items, lab_tests, patient_vitals, admissions, rooms
Inventory	medicines, medicine_inventory
Finance	billing, payments
System	feedback, audit_logs, notifications

Database Review Checklist

- Confirm every table has a primary key.
- Confirm foreign keys reflect intended relationships.
- Add indexes for frequently used join and search columns where appropriate.
- Review uniqueness and nullability constraints.
- Use transactions for multi-table updates.
- Keep secrets out of schema files and source control.
- Use backups and test database restoration.

Appendix note

The database appendix documents the project schema and its key tables. Table definitions and descriptions should match the actual application version submitted with the report.

Appendix B: Source Structure

The source archive is organized as a Maven web project. The following are representative of the supplied structure.

```
src/main/java/com/smarthospital/
  controller/
    AdminServlet.java
    AdmissionServlet.java
    AppointmentServlet.java
    BillingServlet.java
    DoctorServlet.java
    LabServlet.java
    LoginServlet.java
    LogoutServlet.java
    MedicineServlet.java
    NurseServlet.java
    PatientServlet.java
    PrescriptionServlet.java
    ReceptionistServlet.java
  dao/
    PatientDAO.java    DoctorDAO.java    AppointmentDAO.java
    AdmissionDAO.java  BillingDAO.java   LabDAO.java
    MedicineDAO.java   UserDAO.java
  filter/
    AuthenticationFilter.java
    AuthorizationFilter.java
  model/
    Patient.java, Doctor.java, Appointment.java, ...
  service/
    AuthService.java, AppointmentService.java,
    AdmissionService.java, BillingService.java
  util/
    DBConnection.java, PasswordUtil.java, ValidationUtil.java
  exception/
    ApplicationException.java, DatabaseException.java
src/main/resources/db.properties
src/main/webapp/
  admin/ doctor/ patient/ nurse/ receptionist/
  common/ includes/ css/ js/ errors/
database/schema.sql
pom.xml
Dockerfile
```

Build and Configuration Notes

The Maven project declares WAR packaging and Java 17 compilation. Servlet and JSP APIs are marked as provided because the target servlet container supplies them at runtime. Deployment should use a compatible container and should keep production database configuration separate from checked-in files.

Source structure note

The source appendix provides a view of the project organization and representative files. Keep package names, file paths and configuration examples aligned with the submitted codebase.

Appendix C: User Guide

C.1 General Login

- Open the URL in a supported browser.
- Enter the assigned login identifier and password.
- Submit the form and wait for the role-specific dashboard.
- If login fails, check the entered details or contact the system administrator.
- Use the logout control when finished, especially on a shared computer.

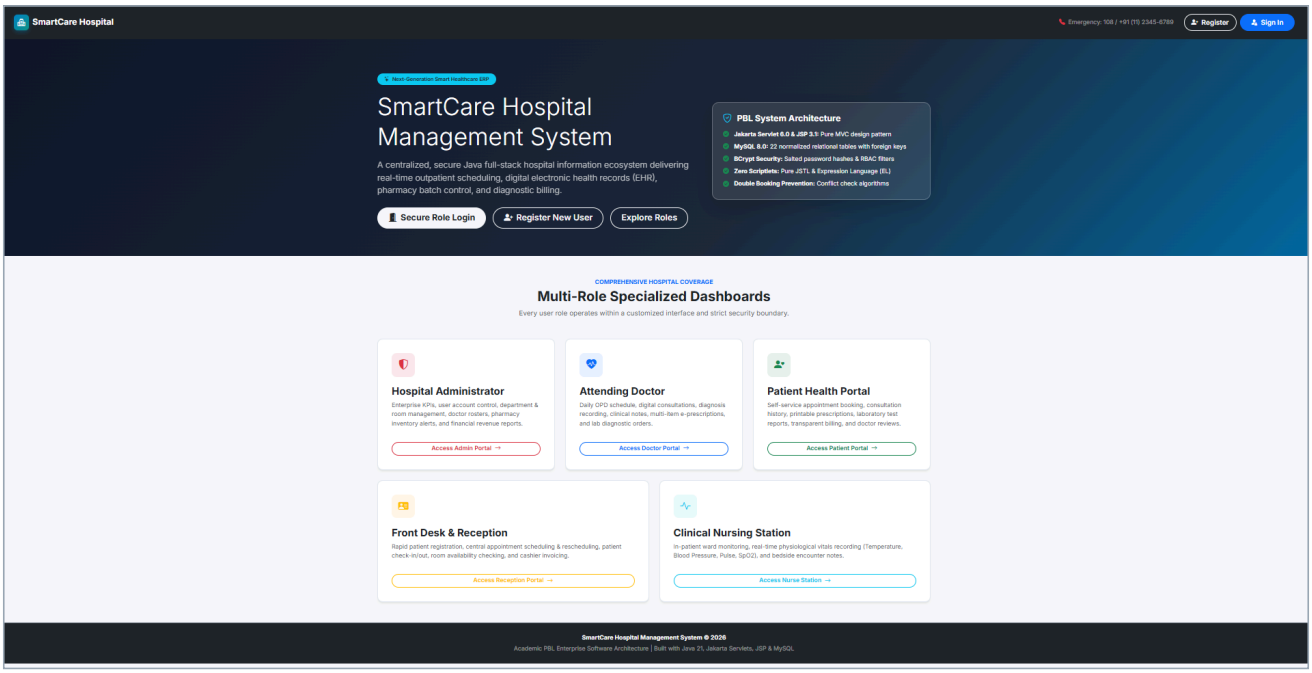


Figure C.1. SmartCare home page with role-based portal cards (open the URL in a browser).

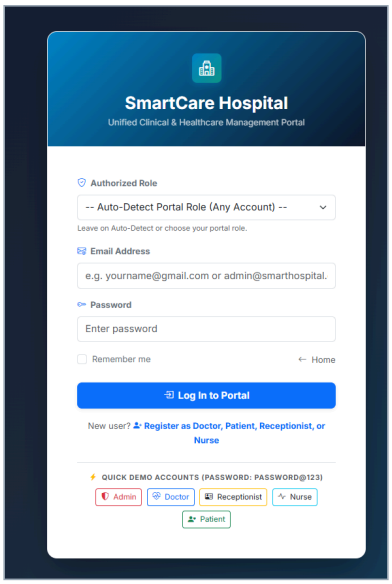


Figure C.2. Login page: select a role or use Auto-Detect, then enter the email and password.

C.2 Administrator

Use the administrator interface to review dashboard summaries and access user, department, doctor, room, billing, report and audit-related screens according to configured permissions. Validate changes carefully because administrative actions can affect multiple workflows.

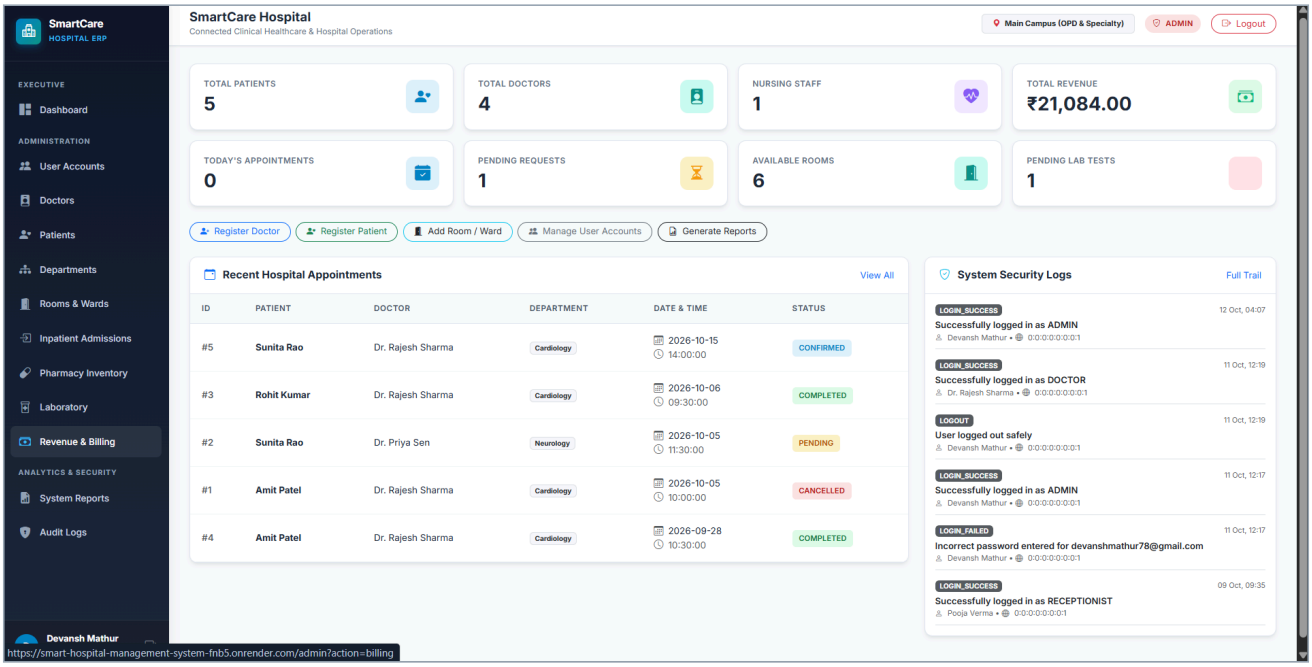


Figure C.3. Administrator dashboard: KPIs, recent appointments and system security logs.

C.3 Patient

Use the patient interface to review the profile and available appointment, medical record, prescription, laboratory report, bill/invoice and feedback pages enabled for the account. Check that displayed records belong to the signed-in patient.

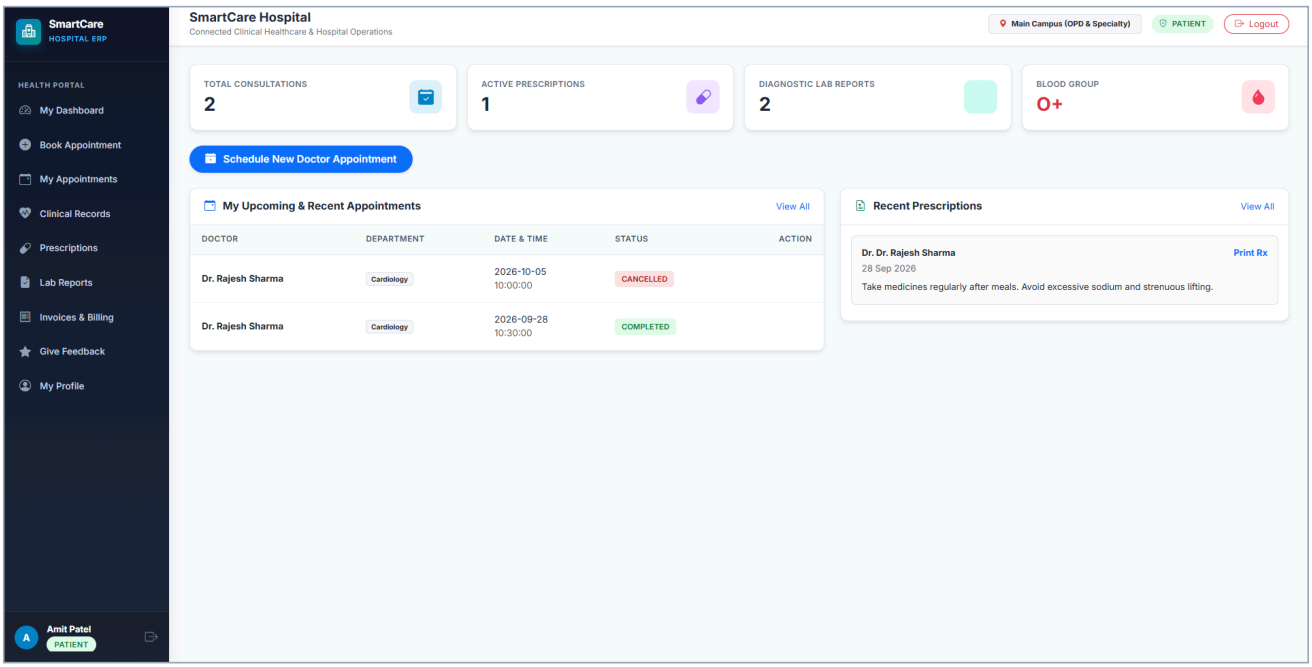


Figure C.4. Patient portal dashboard: consultations, prescriptions, lab reports and appointments.

C.4 Doctor

Use doctor-facing pages to review appointments and patient profiles and to access consultation, record, prescription and laboratory report workflows. Enter clinical information carefully and ensure it is associated with the correct patient.

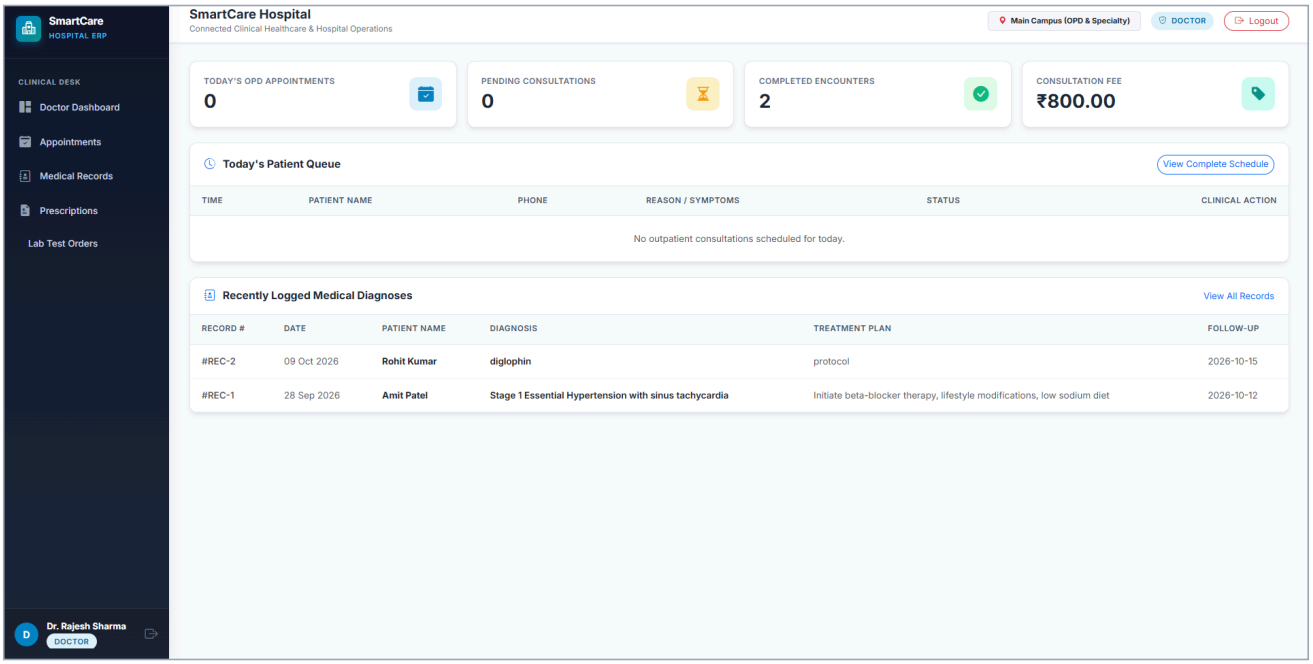


Figure C.5. Doctor dashboard: today's patient queue and recently logged medical diagnoses.

C.5 Nurse

Use nursing pages to view the permitted patient list and record or review vital measurements. Confirm the patient identity before saving measurements.

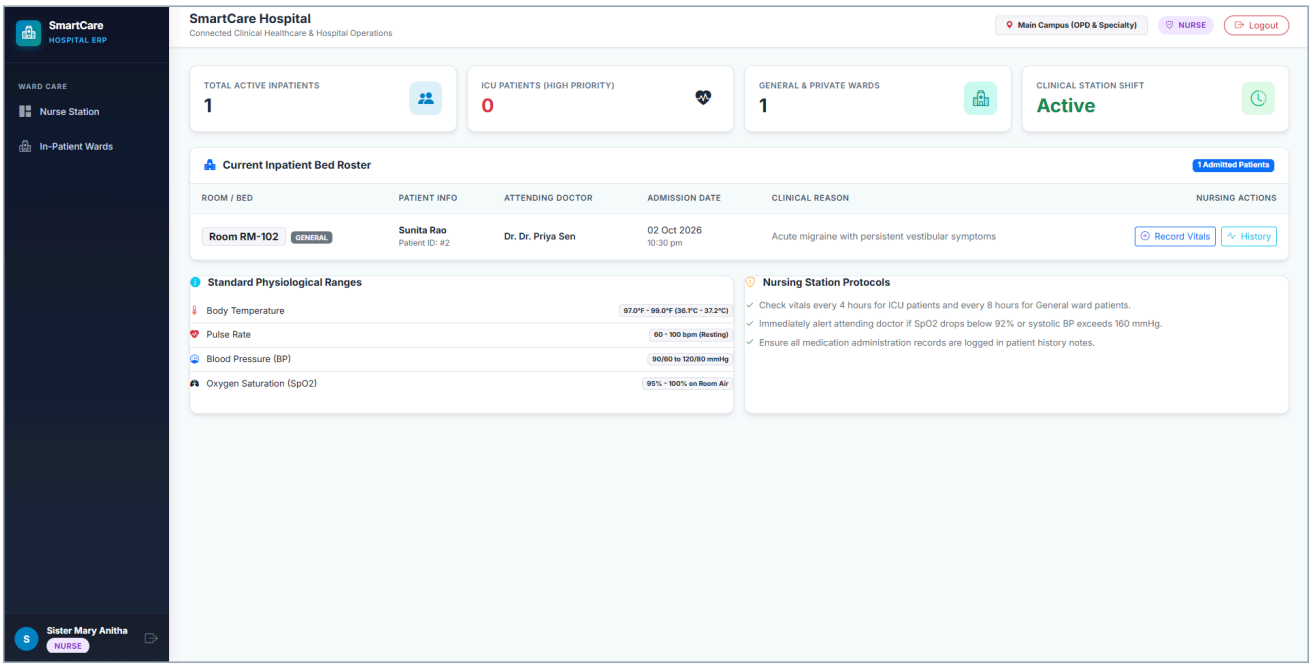


Figure C.6. Nurse station: inpatient bed roster, vitals recording and nursing protocols.

C.6 Receptionist

Use reception pages for patient registration, appointment booking and front-desk workflows such as patient lists, rooms and billing where enabled.

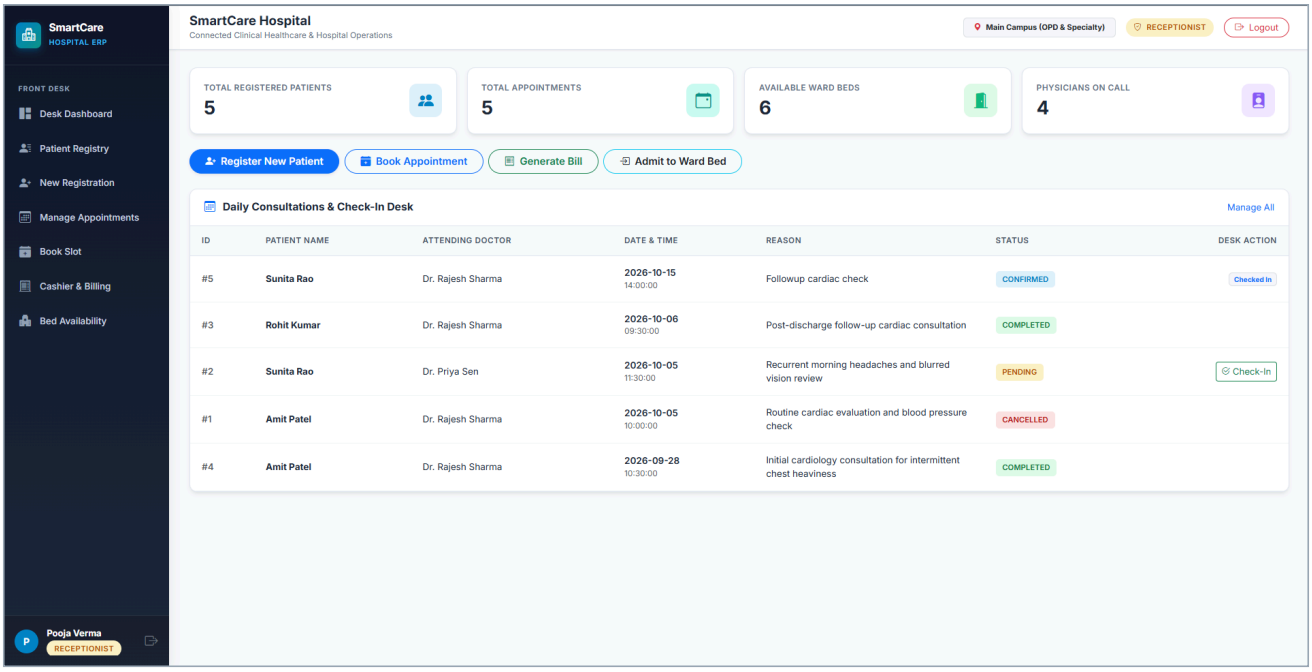


Figure C.7. Front desk dashboard: patient registration, appointment check-in, billing and ward admission.

C.7 Safe Demonstration

For academic demonstration, use test accounts and synthetic patient information. Do not share credentials in screenshots, presentation slides or the report. Verify that logout prevents continued access to protected screens.

User guide summary

The user guide should help each role sign in, navigate its permitted screens and complete routine tasks. Users should follow role restrictions and report unexpected errors rather than bypassing access controls.

Appendix D: Screens and Demonstration Checklist

The source archive contains JSP interfaces for the following screens. The live website could not be fetched successfully during preparation, so the list below is a source-based screen inventory rather than a claim that each screen was captured live.

Role / area	Screens present in source
Common	Login, home/index, navbar, sidebar, header/footer, error pages
Admin	Dashboard, users, departments, doctors, rooms, billing, reports, audit
Doctor	Dashboard, patient prescriptions and reports
Patient	Dashboard, profile, appointments, booking, medical records, prescriptions, lab, invoice, feedback
Nurse	Dashboard, patients, record vitals, vitals view
Receptionist	Dashboard, patient booking, rooms, generate bill
Shared workflows	Admission list/form, inventory, lab management/test details, prescription view, invoice

Suggested Screenshots to Add Before Submission

- Login page with no credentials visible.
- Admin dashboard and user/department management.
- Patient dashboard and appointment booking.
- Doctor appointment/consultation page.
- Nurse vital-recording page using synthetic data.
- Laboratory or medicine inventory page.
- Admission/room management page.
- Billing page with sample data.
- A role-access denial or validation example.

Insert screenshots captured from the working application into this appendix if required by the department. Captures should be legible, use synthetic records and avoid exposing passwords, connection strings or real patient data.

Appendix E: Project Submission Checklist

- Confirm the hosted URL is reachable and the latest build is deployed.
- Run the functional test matrix and replace each 'To execute/confirm' status with evidence-backed results.
- Verify all five roles and direct URL restrictions.
- Confirm database schema and environment variables match the deployed version.
- Remove secrets, private configuration and unnecessary build artifacts from the submitted archive.
- Add current screenshots from the live system if required by the department.
- Verify team names, enrollment numbers, mentor designation and project registration code with official college records.
- Have all team members review and sign the declaration and obtain authorized signatures on the certificate page.

End of report

Final submission checks

Before submission, confirm the hosted build, functional test evidence, role restrictions, database configuration, team details and required signatures. Add current screenshots only when they reflect the verified application.